CINEORBIT
A smartphone Based Landmark-driven Camera Control System

MINI PROJECT REPORT

Submitted by

V.R. Akshay Krishnaa [RA2411003010526]
C.Purnima [RA2411026010582]

Under the Guidance of

Dr.R.Kayalvizhi

21CSE305P – SerBot: Project-Based Learning in Robotics



DEPARTMENT OF NETWORKING AND COMMUNICATIONS

FACULTY OF ENGINEERING AND TECHNOLOGY

SCHOOL OF COMPUTING

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR

NOVEMBER 2026

SRM INSTITUTION OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that the 21CSE305P – SerBot: Project-Based Learning in Robotics course project report titled **"CineOrbit- A smartphone Based Landmark-driven Camera Control"** is the bonafide work done by **V.R.Akshay Krishnaa [RA2411003010526] (Btech,CSE-CORE), C.Purnima [RA2411026010582] (Btech,CSE-AIML) of III Year / V Sem** who carried out the mini project under my supervision.

SIGNATURE
Faculty In-Charge
Dr.R.Kayalvizhi
Associate Professor,
Department of Networking and Communications,
SRM Institute of Science and Technology
Kattankulathur

SIGNATURE
Head of the Department
Dr.Lakshmi M
Professor and Head
Department of Networking and Communications,
SRM Institute of Science and Technology
Kattankulathur

ABSTRACT

Visual landmark tracking plays a crucial role in enabling mobile robots to perceive and interpret their surrounding environments during navigation. However, maintaining continuous landmark visibility becomes challenging when a robot executes directional manoeuvres, as landmarks that are initially suitable for tracking may move outside the camera's field of view or become difficult to observe due to changes in the robot's orientation.

To address this challenge, this project proposes a **Route-Conditioned Predictive Landmark Selection (RCPLS)** algorithm for mobile robots equipped with an independently rotating camera. The proposed approach incorporates predefined route information into the landmark-selection process to anticipate changes in the robot's orientation and predict the future visibility of candidate landmarks before executing upcoming manoeuvres.

The algorithm evaluates candidate landmarks based on multiple factors, including visual tracking reliability, predicted visibility along the anticipated trajectory, tracking persistence and camera rotational constraints. By integrating these parameters into a landmark suitability assessment, the system identifies and prioritizes landmarks that are expected to remain observable throughout upcoming movements. Furthermore, it enables proactive landmark switching when the currently selected landmark is predicted to become unsuitable for continued tracking.

The proposed approach aims to overcome the limitations of purely reactive landmark-selection strategies by incorporating future navigation conditions into the decision-making process. Through route-aware visibility prediction and adaptive landmark selection, the project seeks to improve visual tracking continuity, reduce unexpected tracking interruptions and enhance the reliability of visual perception during mobile robot navigation.

ACKNOWLEDGEMENT

We express our heartfelt thanks to our honourable **Vice Chancellor Dr. C. MUTHAMIZHCHELVAN**, for being the beacon in all our endeavours. We would like to express my warmth of gratitude to our **Registrar Dr. S Ponnusamy,** for his encouragement.

We express our profound gratitude to our **Dean (CET) Dr. Leenus Jesu Martin M,** for bringing out novelty in all executions. We would like to express my heartfelt thanks to Chairperson, School of Computing **Dr. Revathi Venkataraman,** for imparting confidence to complete my course project. We would like to express my thanks to Associate Chairperson, School of Computing **Dr. Pushpalatha M,** for complete of my course project.

We extend my gratitude to our **HOD Dr. Lakshmi M, Professor and Head, Department of Networking and Communications** and Departmental colleagues for their Support.

We wish to express my sincere thanks to **Course Audit Professors Dr.G.Usha, Professor, Department of Computing Technology** and **Course Coordinator** for their constant encouragement and support.

We are highly thankful to our Course project Faculty **Dr.Kayalvizhi R, Associate Professor, Department of Networking and Communications,** for her assistance, timely suggestion and guidance throughout the duration of this course project.

Finally, we thank our parents and friends near and dear ones who directly and indirectly contributed to the successful completion of our project. Above all, I thank the almighty for showering his blessings on me to complete my Course project.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
1.	Introduction	4
2.	Literature Survey	6
3.	Requirement Analysis 3.1 Hardware 3.2 Software	9
4.	Architecture and Design 4.1 Circuit diagram, 4.2 PIN diagram, 4.3 Module explanation	13
5.	Implementation	17
6.	Experiment Result and Analysis	32
7.	Conclusion	34
8.	Future Scope	35
9.	References	36

CHAPTER 1
INTRODUCTION

Mobile robotics has become an important area of technological advancement, with applications in autonomous navigation, industrial automation, surveillance and warehouse management. The ability of a mobile robot to perceive its surroundings and navigate efficiently depends on its visual perception capabilities. Computer vision enables robots to capture and interpret environmental information through cameras, while visual landmark tracking helps identify and continuously monitor distinctive features in the surroundings.

However, maintaining consistent landmark tracking becomes challenging when a robot performs directional manoeuvres such as left and right turns. A landmark that is clearly visible during straight movement may move outside the camera's field of view when the robot changes direction, resulting in tracking interruptions. Conventional landmark-selection approaches primarily consider current visual quality and tracking reliability, without necessarily accounting for upcoming movements.

To address this challenge, this report proposes Route-Conditioned Predictive Landmark Selection (RCPLS), a framework that incorporates planned camera trajectories into the evaluation and selection of visual landmarks. The proposed approach considers multiple characteristics of candidate landmarks, including feature quality, feature availability, spatial distribution, current visibility, and temporal consistency. In addition, it introduces a route-conditioned predictive component that evaluates the expected visibility and tracking suitability of landmarks with respect to the intended camera movement. Based on these evaluations, the framework selects a primary landmark while retaining alternative landmarks as backups to support recovery when the primary landmark becomes unreliable or is lost.

The proposed framework combines established computer vision techniques, including Shi-Tomasi corner detection for candidate feature identification, ORB for feature description, and Lucas-Kanade optical flow for inter-frame tracking. These techniques form the feature detection and tracking foundation, while the route-conditioned ranking and predictive selection strategy constitute the central focus of RCPLS. The framework is designed to support continuous visual tracking in robotic camera platforms by considering both present visual conditions and anticipated changes resulting from camera motion.

The primary objective of this work is to investigate whether incorporating planned camera trajectories into landmark selection can improve landmark availability, tracking continuity, and recovery from feature loss compared with conventional feature-selection approaches. The framework is considered in the context of robotic cinematography, where maintaining reliable visual references during controlled camera movements is important for achieving consistent and repeatable motion. Its effectiveness can be assessed through comparative experiments involving different camera trajectories, environmental conditions, and landmark visibility changes.

CHAPTER 2

LITERATURE SURVEY

Visual landmark detection and camera motion control are important aspects of computer vision-based robotic systems. Research in these areas has explored feature detection, image tracking, visual servoing, embedded controllers and wireless communication. These techniques provide the foundation for developing CineOrbit, which integrates smartphone-based landmark detection with an ESP32-controlled motorized camera platform.

The following literature survey discusses the major techniques and technologies relevant to the proposed system.

2.1 Visual Landmark Detection

Shi and Tomasi (1994), in their paper Good Features to Track, proposed a technique for identifying reliable image features that can be tracked across consecutive frames. Their approach evaluates image points based on their suitability for tracking and is particularly useful for identifying distinctive corners. This work provides a foundation for detecting stable visual landmarks using the smartphone camera.

2.2 Feature Detection and Description Using ORB

Rublee et al. (2011), in ORB: An Efficient Alternative to SIFT or SURF, introduced ORB (Oriented FAST and Rotated BRIEF), a computationally efficient feature detection and description technique. It supports fast feature extraction and matching while maintaining rotation robustness. Its relatively low computational requirements make it suitable for smartphone-based computer vision applications. In CineOrbit, ORB can be used to identify distinctive visual features and establish candidate landmarks for tracking.

2.3 Optical Flow-Based Landmark Tracking

Lucas and Kanade (1981), in An Iterative Image Registration Technique with an Application to Stereo Vision, introduced an image-registration technique that estimates the movement of features between consecutive image frames using intensity gradients. The Lucas–Kanade method provides an effective foundation for tracking selected landmarks as the camera or its platform changes orientation. This is relevant to maintaining visual tracking while the camera rotates.

2.4 Visual SLAM and Feature-Based Tracking – Classical Framework Review

Research on Visual Simultaneous Localization and Mapping (V-SLAM) has established the importance of visual feature detection, matching, keyframe selection, uncertainty estimation and map representation in autonomous robotic systems. A review of classical V-SLAM frameworks highlights how feature-based methods support camera localization and environmental mapping. These studies provide a broader foundation for understanding the relationship between visual feature reliability and robotic motion estimation. However, the objectives of complete SLAM systems differ from the specific landmark-selection problem considered in RCPLS.

2.5 Keyframe Selection for Visual Localization and Mapping – Systematic Literature Review

The systematic literature review Keyframe Selection for Visual Localization and Mapping Tasks (2023) examines different approaches used to select informative frames in visual localization and mapping systems. It discusses the importance of maintaining sufficient visual coverage while controlling computational requirements. The review also identifies limitations in the theoretical formulation and experimental validation of existing keyframe-selection approaches. Although keyframe selection and landmark selection are different problems, this work highlights the broader importance of selecting reliable visual information for continuous tracking and localization.

2.6 ESP32-Based Embedded Motor Control

The ESP32 technical documentation published by Espressif Systems describes the controller's integrated Wi-Fi connectivity, GPIO interfaces and PWM capabilities. These features make it suitable for wireless communication and motor-control applications. In CineOrbit, the ESP32 serves as the hardware controller, receiving movement commands from the smartphone and controlling the motorized camera platform through a motor driver.

2.7 Research Gap

Existing research provides established methods for visual feature detection, feature description and inter-frame tracking. Shi-Tomasi identifies suitable corner features, ORB provides efficient feature descriptions, and Lucas-Kanade optical flow estimates feature displacement between consecutive frames. These techniques form a strong foundation for visual tracking systems.

However, a potential limitation arises when these techniques are applied to robotic camera platforms executing predefined trajectories. Conventional feature-selection approaches primarily evaluate the visual suitability of landmarks using information available in the current frame. Although feature-monitoring methods can detect occlusion and tracking failure, this does not necessarily mean that the selection process anticipates whether a landmark will remain visible during a known future camera movement.

Based on the identified gaps, this work proposes Route-Conditioned Predictive Landmark Selection (RCPLS), a framework that incorporates planned camera trajectories into visual landmark selection. The proposed framework combines Shi-Tomasi corner detection for candidate landmark identification, ORB feature descriptors for landmark description, A weighted ranking mechanism for evaluating candidate landmarks, Route-conditioned prediction for estimating future landmark visibility, Lucas-Kanade optical flow for tracking selected landmarks and Primary and backup landmark management for tracking recovery.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 CURRENTLY EXISTING SYSTEM

Existing motorized camera platforms generally provide basic camera rotation through manual controls or predefined movement commands. These systems allow users to adjust camera direction using motors, but they may not incorporate intelligent visual landmark detection to determine and maintain a suitable viewing direction.

Similarly, conventional visual tracking systems use computer vision techniques to detect and track objects or features in camera images. However, these systems may not integrate landmark-based tracking with programmable camera movement and route-aware visibility prediction.

Limitations of the Existing System:

- Camera movement is primarily controlled manually or through predefined commands.
- Limited integration of visual landmark detection with motorized camera control.
- Lack of predictive landmark selection based on upcoming movements.
- Tracking interruptions may occur when selected landmarks move outside the camera's field of view.
- Limited coordination between visual perception and camera movement.

3.2 PROJECT REQUIREMENT

Requirement analysis for CineOrbit involves identifying the hardware and software components required to develop a smartphone-controlled programmable camera-motion platform with landmark-based detection.

The system integrates smartphone-based computer vision with an ESP32-controlled motorized platform. It enables wireless camera movement, visual landmark detection, tracking and intelligent camera-direction adjustment.

The requirements are categorized into functional requirements, non-functional requirements, hardware requirements and software requirements.

Functional Requirements

1. Camera Input: The system shall capture environmental images using the smartphone camera.
2. Landmark Detection: The system shall identify distinctive visual landmarks from captured images.
3. Landmark Tracking: The system shall track selected landmarks across consecutive frames.
4. Landmark Selection: The system shall evaluate candidate landmarks and select suitable landmarks for continued tracking.
5. Route-Based Prediction: The system shall use predefined movement information to anticipate upcoming directional changes.
6. Visibility Estimation: The system shall estimate whether selected landmarks will remain within the camera's field of view.
7. Wireless Communication: The smartphone shall communicate with the ESP32 through a local Wi-Fi connection.
8. Motor Control: The ESP32 shall control the motor direction through the motor driver.
9. Programmable Movement: The system shall support timed directional movements, pauses and rotational actions.
10. Movement Control: The user shall be able to start, pause, resume and stop camera movement.

Non-Functional Requirements

1. Performance: The system should process camera images and generate movement commands with minimal delay.
2. Reliability: The system should maintain consistent communication between the smartphone and ESP32.
3. Accuracy: Landmark detection and tracking should provide reliable visual references.
4. Usability: The web-based interface should provide simple and accessible camera controls.

5. Scalability: The system should support additional landmark-selection and tracking functionalities.
6. Efficiency: The system should utilize smartphone processing capabilities effectively.
7. Safety: Motor operations should include appropriate stopping mechanisms to prevent unintended movement.
8. Maintainability: The software modules should be designed to support future improvements and integration.

Table 1. Hardware components used to make the rower

Hardware component	**Purpose**
ESP32 development board	Main controller for wireless communication and motor control
Smartphone	Captures images and processes visual landmarks
L298N motor driver	Controls motor direction
3.5 RPM geared DC motor	Provides rotational movement to the camera platform
12V DC power supply	Supplies power to the motor circuit
LM2596 buck converter	Regulates voltage for the controller
Camera mounting platform	Supports the smartphone during rotation
Connecting wires	Establishes electrical connections

Table 2. Software Tools and Technologies Used in the Development of CineOrbit and Their Purposes

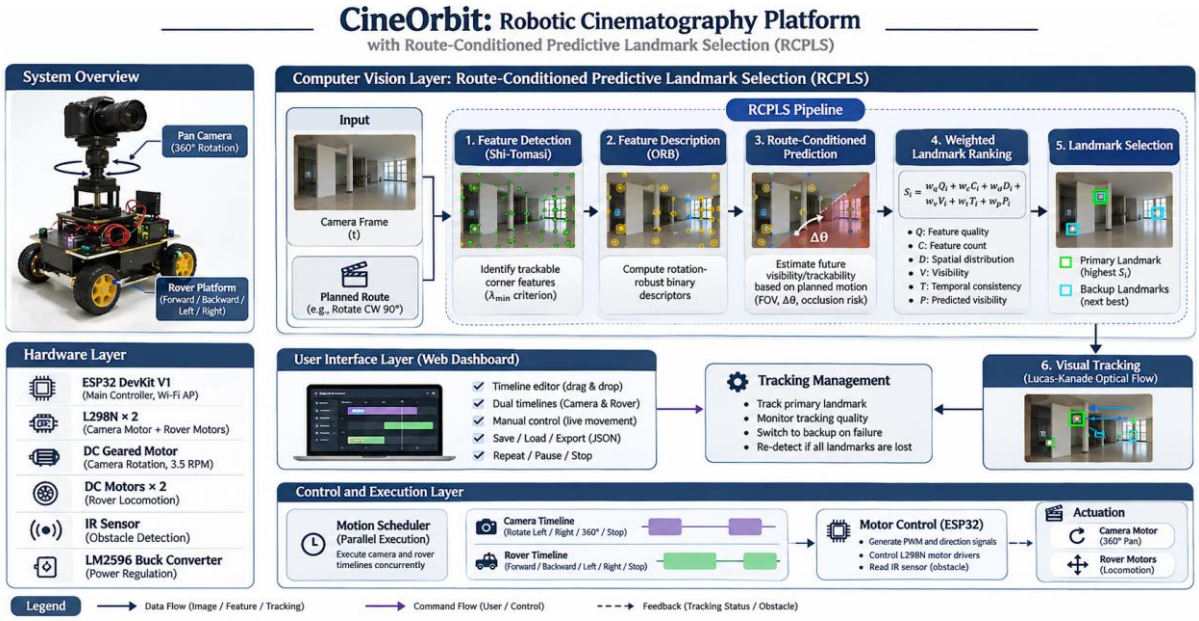
Software / Technology	**Purpose**
Arduino IDE	Programming and uploading ESP32 firmware
ESP32 Arduino Core	Provides ESP32 development support
Python	Computer vision and image-processing development, if used
OpenCV	Landmark detection and tracking
HTML	Structure of the web-based control interface
CSS	Styling and layout of the interface
JavaScript	Timeline editor and movement-control interactions
Wi-Fi	Wireless communication between smartphone and ESP32
HTTP	Transmitting motor-control commands
JSON	Importing and exporting programmed movement sequences
Web Browser	Accessing and controlling the CineOrbit platform

CHAPTER 4
ARCHITECTURE AND DESIGN

Designing the architecture and user interface for a GUI-based Alumni Connect Project using Python requires careful planning to ensure the system's functionality, usability, and scalability. Below is a high-level architecture and design overview for such a project:

4.1 Architecture: The CineOrbit system consists of a smartphone camera, an image-processing module, an ESP32 microcontroller, and a motorized camera platform. The smartphone captures environmental images, which are processed using JavaScript and OpenCV.js to detect visual landmarks and features. Based on the analysis, directional commands are generated and transmitted to the ESP32 through Wi-Fi communication. The ESP32 controls the motor driver, which operates the geared DC motor to rotate the camera platform. This process enables continuous visual analysis and camera alignment.

4.1.1 Architecture Diagram



4.1.1: System Architecture Diagram of the CineOrbit Project.

4.1.2 UML Diagram

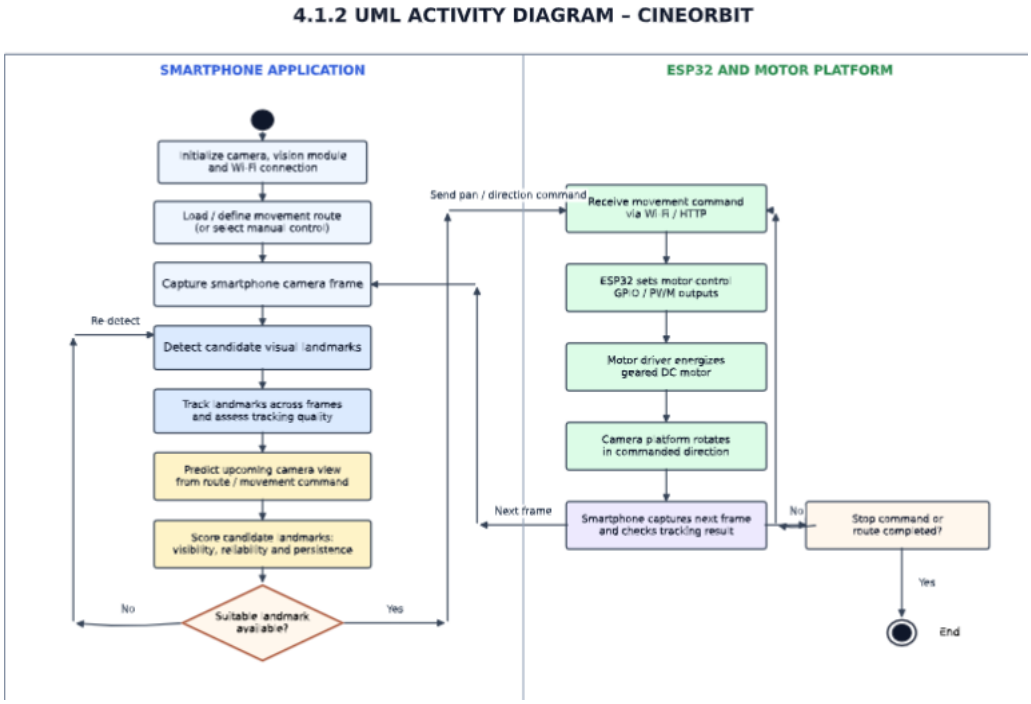


Figure 4.1.2: UML Activity Diagram Illustrating the Workflow of the CineOrbit System

4.1.3 Use Case Diagram

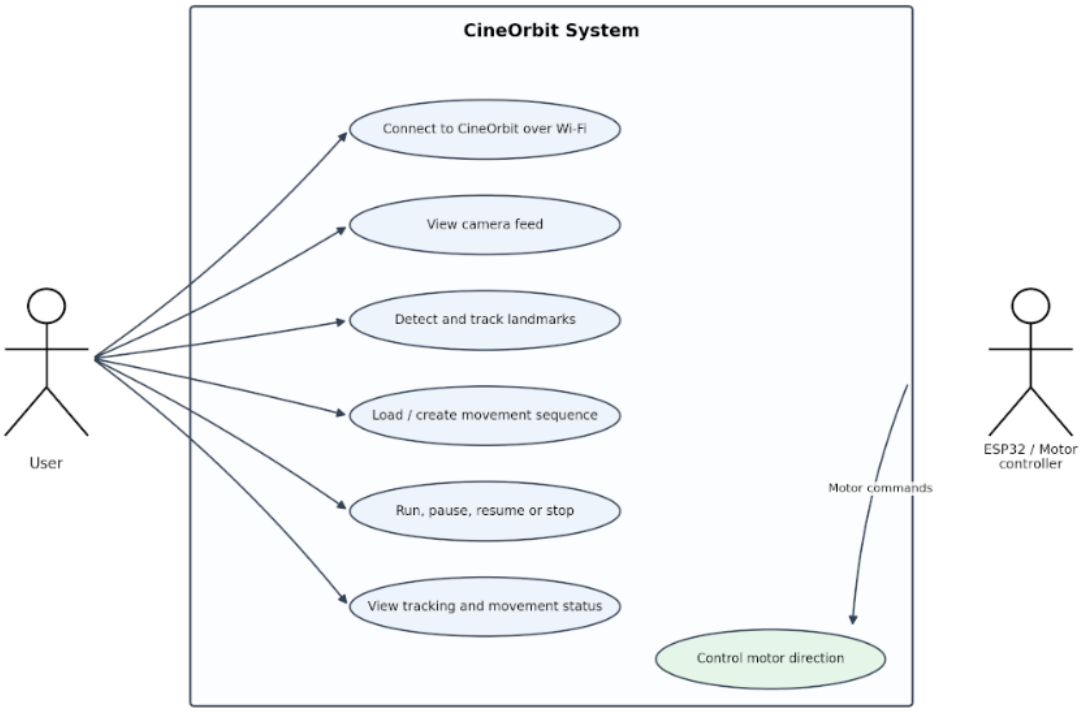


Figure 4.1.3: Use Case Diagram Representing User Interactions with the CineOrbit System.

4.1.4 Circuit Diagram

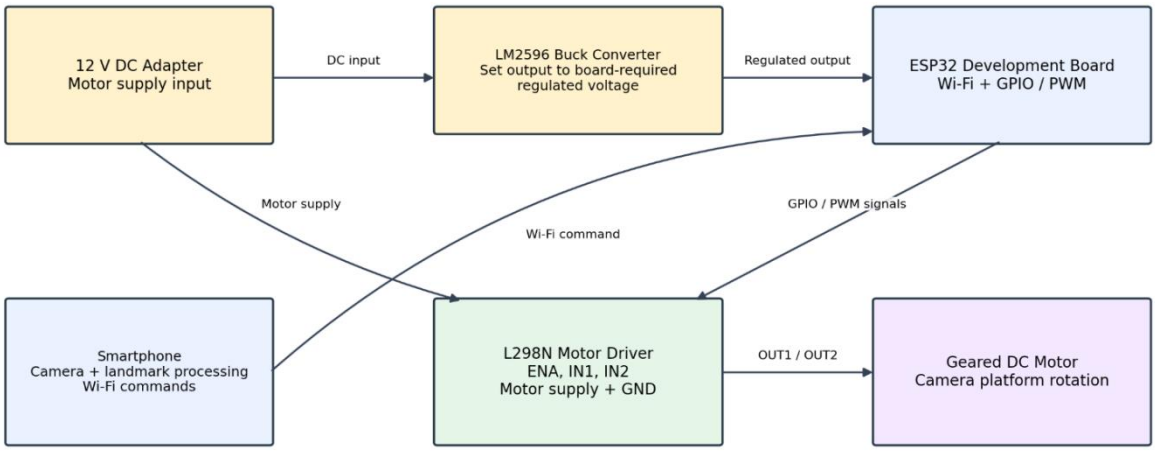


Figure 4.1.4: Circuit Diagram Showing the Hardware Connections Between the ESP32, Motor Driver, Power Supply, and Geared DC Motor.

4.1.5 PIN Diagram

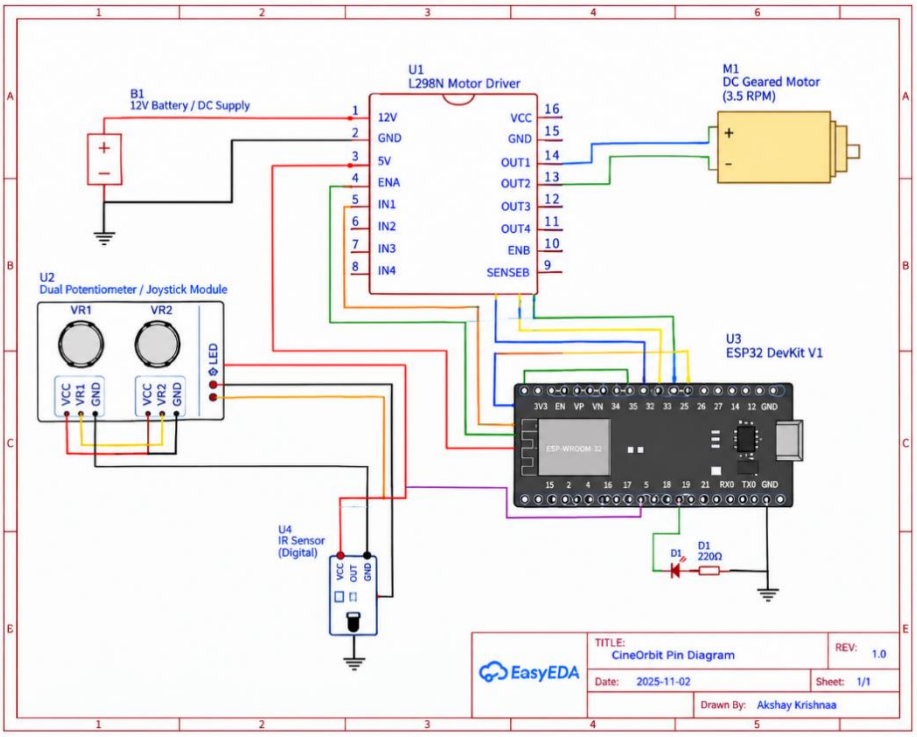


Figure 4.1.5: Pin Diagram showing detailed pin connections between components.

4.1.6 Explanation of Diagrams

Figure1.4.1: The overall system architecture illustrates the integration of the computer vision module, user interface, hardware components and motor-control subsystem. The computer vision layer implements the proposed RCPLS framework, which uses Shi-Tomasi for landmark detection, ORB for feature description, route-conditioned prediction for estimating future landmark visibility, and weighted ranking for selecting primary and backup landmarks. Lucas-Kanade optical flow is used to track the selected landmarks across consecutive frames.

Figure1.4.2: The UML activity diagram represents the sequence of operations performed by CineOrbit, divided into two swimlanes: the Smartphone Application and the ESP32 and Motor Platform. The process begins with camera and Wi-Fi initialization, followed by movement-route selection and camera-frame acquisition. The application detects candidate landmarks, tracks them across frames, evaluates their reliability and predicts their future visibility based on the planned camera movement. If suitable landmarks are unavailable, the system repeats the detection process; otherwise, it generates and transmits movement commands to the ESP32.

Figure1.4.3: The UML use case diagram illustrates the interaction between the user, CineOrbit system and ESP32 motor controller. The user can connect to CineOrbit through Wi-Fi, view the camera feed, initiate landmark detection and tracking, create or load movement sequences, and control playback through run, pause, resume and stop operations. The interface also allows the user to monitor tracking and movement status.

Figure1.4.4: The hardware block diagram illustrates the major components and their communication and power connections. A 12 V DC supply provides power to the motor-control system, while the LM2596 buck converter regulates the voltage required by the electronic components. The smartphone performs camera-frame acquisition and vision processing, generating movement commands that are transmitted to the ESP32 through Wi-Fi.

Figure1.4.5: The circuit schematic represents the electrical connections between the ESP32 DevKit V1, L298N motor driver, geared DC motor, power supply, joystick module, IR sensor and LED. The ESP32 communicates with the motor driver through GPIO and PWM signals to control motor direction and speed. The joystick module provides manual input, while the

IR sensor supplies digital obstacle-detection signals.

CHAPTER 5

IMPLEMENTATION DETAILS

Implementing the CineOrbit project involves developing the smartphone-based computer vision system, programming the ESP32 microcontroller, establishing wireless communication and integrating the motorized camera platform. The implementation is divided into the following major stages.

1. Set Up the Development Environment

a. Configure the ESP32: Install the Arduino IDE and ESP32 board support package to program the microcontroller and configure its GPIO pins for motor control.

b. Set Up the Smartphone: Configure the smartphone camera and development environment for capturing images and processing visual information.

c. Install Required Libraries: Use suitable computer vision libraries and frameworks for image processing, landmark detection and tracking.

d. Establish Wireless Communication: Configure the ESP32 as a local Wi-Fi access point to enable communication between the smartphone and the motorized camera platform.

2. Design the System Architecture

a. Smartphone Processing Module: Design the image-processing workflow to capture camera frames, detect visual landmarks and track selected features.

b. Landmark Selection Module: Develop the algorithm to evaluate candidate landmarks based on tracking reliability, predicted visibility and upcoming camera movements.

c. Motor Control Module: Configure the ESP32 to receive movement commands and control the geared DC motor through the L298N motor driver.

d. Communication Module: Establish HTTP-based communication between the smartphone interface and ESP32 for transmitting movement commands and control instructions.

3. Develop the User Interface

a. Design the Web Interface: Develop a browser-based interface using HTML, CSS and JavaScript to control the camera platform through a smartphone.

b. Implement Movement Controls: Provide controls for rotating the camera left, rotating right, stopping and performing 360-degree movements.

c. Develop the Timeline Editor: Create a visual timeline editor that allows users to arrange, reorder and configure timed camera movements.

d. Implement Playback Controls: Provide Run, Pause, Resume, Stop and Repeat functionalities for executing programmed movement sequences.

e. Implement Sequence Management: Enable users to save movement sequences using browser local storage and import or export sequences through JSON files.

4. Implement Landmark Detection and Tracking

a. Image Acquisition: Capture environmental images using the smartphone camera.

b. Image Preprocessing: Prepare captured frames for feature detection through resizing, grayscale conversion and noise reduction, where required.

c. Landmark Detection: Identify distinctive visual features from camera frames using suitable computer vision techniques.

d. Landmark Tracking: Track selected landmarks across consecutive frames to monitor their movement and visibility.

e. Predictive Landmark Selection: Evaluate candidate landmarks using predefined route information and predicted visibility to identify suitable landmarks for continued tracking.

f. Camera Movement Adjustment: Generate appropriate directional commands and transmit them to the ESP32 to adjust the camera platform.

5. Integrate Hardware and Software

a. ESP32 Programming: Upload the firmware responsible for Wi-Fi communication and motor-control operations.

b. Motor Driver Integration: Connect the ESP32 to the L298N motor driver to control the direction of the geared DC motor.

c. Power Supply Integration: Connect the motor supply and regulated power circuit while ensuring appropriate voltage levels.

d. System Integration: Connect the smartphone-based vision system with the ESP32-controlled motor platform to coordinate landmark tracking and camera movement.

6. Testing and Validation

a. Camera Testing: Verify smartphone camera input and image acquisition.

b. Landmark Detection Testing: Evaluate the detection and tracking of visual landmarks under different viewing conditions.

c. Motor Control Testing: Verify the directional movement and stopping functionality of the camera platform.

d. Wireless Communication Testing: Validate command transmission between the smartphone and ESP32.

e. Integrated System Testing: Evaluate camera movement and landmark tracking during different directional changes.

CODE

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

#define ENA D5
#define IN1 D6
#define IN2 D7

const char* AP_NAME = "CineOrbit";
const char* AP_PASSWORD = "orbit123";

const int MOTOR_SPEED = 1023;
const unsigned long COMMAND_TIMEOUT = 700;

ESP8266WebServer server(80);
unsigned long lastCommandTime = 0;
bool motorRunning = false;

const char PAGE[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>CineOrbit Studio V2</title>
<style>
*{box-sizing:border-box}
body{margin:0;padding:16px;background:#10141c;color:#f1f5f9;
font-family:Arial,sans-serif}
h1{margin-bottom:4px}
.sub,small{color:#aab8c9}
.panel{background:#1b2330;padding:15px;border-radius:14px;margin:14px 0}
button{border:0;border-radius:8px;padding:11px;font-weight:bold;cursor:pointer}
button:disabled{opacity:.4}
.row{display:flex;gap:8px;flex-wrap:wrap;align-items:center}
.row button{flex:1}
.primary{background:#3182ce;color:white}
.secondary{background:#394454;color:white}
.danger{background:#d94343;color:white}
.actions{display:grid;grid-template-columns:1fr 1fr;gap:8px}
.action{background:#29384d;color:white;text-align:left}
input,select{background:#10141c;color:white;border:1px solid #536176;
padding:9px;border-radius:7px;max-width:100%}
input[type=number]{width:95px}
input[type=checkbox]{width:18px;height:18px;vertical-align:middle}
.timelineScroll{overflow-x:auto;border:1px solid #46546a;border-radius:10px}
.track{position:relative;min-width:100%;width:max-content;padding-bottom:12px}
.ruler{height:38px;position:relative;border-bottom:1px solid #536176;
background:#202a39}
.tick{position:absolute;top:0;height:100%;border-left:1px solid #536176;
padding:5px 0 0 4px;color:#b9c5d4;font-size:11px}
.lane{position:relative;min-height:90px;padding:10px;
background:#151c27}
.block{position:absolute;top:12px;height:64px;border-radius:8px;
padding:6px;color:white;cursor:grab;overflow:hidden;
border:2px solid #78a5dc;background:#2b5685;min-width:58px}
.block.stop{background:#66512b;border-color:#c8a65e}
```

```
.block.turn{background:#563d79;border-color:#aa8de0}
.block.active{outline:3px solid #f5d76e}
.block strong{font-size:12px;display:block;white-space:nowrap}
.block small{font-size:11px;white-space:nowrap;color:#fff}
#status{color:#81d8a5}
</style>
</head>
<body>
<h1>CineOrbit Studio</h1>
<p class="sub">Timeline V2 · Motion sequencing</p>

<div class="panel">
<h3>Action library</h3>
<p><small>Drag an action onto the timeline, or tap to append it.</small></p>
<div class="actions">
<button class="action" draggable="true" data-type="left">↶ Rotate Left</button>
<button class="action" draggable="true" data-type="right">↷ Rotate Right</button>
<button class="action" draggable="true" data-type="stop">II Stop / Wait</button>
<button class="action" draggable="true" data-type="turn">⟳ 360° Rotation</button>
</div>
</div>

<div class="panel">
<h3>Timeline ruler</h3>
<p><small>Blocks are proportional to duration. Drag blocks to reorder them.
The ruler scrolls horizontally for longer sequences.</small></p>
<div class="timelineScroll" id="timelineScroll">
<div class="track" id="track">
<div class="ruler" id="ruler"></div>
<div class="lane" id="lane"></div>
</div>
</div>
<p>Total duration: <strong id="total">0.0 seconds</strong></p>
<p><small>Select a block to edit its duration or 360° direction below.</small></p>
<div id="editor"><small>No action selected.</small></div>
<div class="row" style="margin-top:12px">
<button class="secondary" onclick="duplicateSelected()">Duplicate</button>
<button class="secondary" onclick="deleteSelected()">Delete selected</button>
<button class="secondary" onclick="clearTimeline()">Clear timeline</button>
</div>
</div>

<div class="panel">
<h3>360° calibration</h3>
<label>Duration (seconds)
<input id="turnSeconds" type="number" min="1" max="120" step="0.1" value="17.1">
</label>
<p><small>17.1 seconds is a starting estimate for a 3.5 RPM motor. Calibrate with your platform
installed.</small></p>
</div>

<div class="panel">
<h3>Playback</h3>
<label><input id="repeat" type="checkbox"> Repeat timeline continuously</label>
<div class="row" style="margin-top:12px">
<button class="primary" onclick="runTimeline()">▶ Run</button>
<button class="secondary" onclick="pauseTimeline()">II Pause / Resume</button>
```

```
<button class="danger" onclick="stopTimeline()">■ Stop</button>
</div>
<p id="status">Ready</p>
<p id="progress">No sequence running.</p>
</div>

<div class="panel">
<h3>Save and exchange sequences</h3>
<input id="sequenceName" placeholder="Sequence name" style="width:100%">
<div class="row" style="margin-top:10px">
<button class="primary" onclick="saveSequence()">Save</button>
<button class="secondary" onclick="loadSequence()">Load saved</button>
<button class="danger" onclick="deleteSequence()">Delete saved</button>
</div>
<select id="savedList" style="width:100%;margin-top:10px">
<option value="">Choose saved sequence</option>
</select>
<hr style="border-color:#394454;margin:18px 0">
<div class="row">
<button class="primary" onclick="exportJSON()">Export JSON</button>
<button class="secondary" onclick="document.getElementById('importFile').click()">Import
JSON</button>
<input id="importFile" type="file" accept=".json,application/json" hidden>
</div>
<p><small>Export downloads a file you can keep or share. Import replaces the current editor sequence
after validation.</small></p>
</div>

<script>
const $ = id => document.getElementById(id);
const track = $('track'), ruler = $('ruler'), lane = $('lane');

let steps = [];
let nextId = 1;
let selectedId = null;
let running = false, paused = false, stopRequested = false;
let dragData = null, activeId = null;
const PX_PER_SECOND = 60;

function label(type) {
  return ({left:'Rotate Left',right:'Rotate Right',
    stop:'Stop / Wait',turn:'360° Rotation'})[type];
}
function durationOf(s) {
  return s.type === 'turn'
    ? Math.max(1,Number($('turnSeconds').value)||17.1)
    : Math.max(.1,Number(s.duration)||.1);
}
function totalDuration(list=steps) {
  return list.reduce((sum,s)=>sum+durationOf(s),0);
}
function newStep(type) {
  const defaults = {left:3,right:3,stop:2,turn:17.1};
  return {id:nextId++,type,duration:defaults[type],direction:'right'};
}
function addStep(type,index=steps.length) {
  if(running)return;
```

```
  const s=newStep(type);
  steps.splice(index,0,s);
  selectedId=s.id;
  render();
}
function setStatus(s){$('status').textContent=s;}

function render() {
  const total=totalDuration();
  const width=Math.max(420,total*PX_PER_SECOND+20);
  track.style.width=width+'px';
  ruler.style.width=width+'px';
  lane.style.width=width+'px';
  ruler.innerHTML='';
  lane.innerHTML='';

  const tickEvery=total>100?10:(total>40?5:1);
  for(let t=0;t<=total+0.001;t+=tickEvery){
    const tick=document.createElement('div');
    tick.className='tick';
    tick.style.left=(t*PX_PER_SECOND)+'px';
    tick.textContent=t+'s';
    ruler.appendChild(tick);
  }

  let cursor=0;
  steps.forEach((s,index)=>{
    const dur=durationOf(s);
    const block=document.createElement('div');
    block.className='block '+s.type+
      (selectedId===s.id?' selected':'')+
      (activeId===s.id?' active':'');
    block.style.left=(cursor*PX_PER_SECOND+2)+'px';
    block.style.width=Math.max(54,dur*PX_PER_SECOND-4)+'px';
    block.draggable=!running;
    block.innerHTML='<strong></strong><small></small>';
    block.children[0].textContent=(index+1)+'. '+label(s.type);
    block.children[1].textContent=
      (s.type==='turn'?s.direction+' · ':'')+dur.toFixed(1)+' s';

    block.onclick=()=>{selectedId=s.id;render();};
    block.addEventListener('dragstart',e=>{
      if(running){e.preventDefault();return;}
      dragData={kind:'existing',id:s.id};
      e.dataTransfer.setData('text/plain','existing:'+s.id);
    });
    block.addEventListener('dragover',e=>e.preventDefault());
    block.addEventListener('drop',e=>{
      e.preventDefault();e.stopPropagation();
      dropOnStep(s.id);
    });
    lane.appendChild(block);
    cursor+=dur;
  });

  $('total').textContent=total.toFixed(1)+' seconds';
  renderEditor();
```

```
}

function renderEditor(){
  const editor=$('editor');
  editor.innerHTML='';
  const s=steps.find(x=>x.id===selectedId);
  if(!s){editor.innerHTML='<small>No action selected.</small>';return;}

  const title=document.createElement('p');
  title.textContent=label(s.type);
  editor.appendChild(title);

  if(s.type==='turn'){
    const select=document.createElement('select');
    [['right','Clockwise / Right'],['left','Counter-clockwise / Left']]
      .forEach(([v,t])=>{
        const o=document.createElement('option');
        o.value=v;o.textContent=t;select.appendChild(o);
      });
    select.value=s.direction;
    select.disabled=running;
    select.onchange=()=>{s.direction=select.value;render();};
    editor.appendChild(select);
    const info=document.createElement('p');
    info.innerHTML='<small>Uses the 360° calibration duration above.</small>';
    editor.appendChild(info);
  }else{
    const input=document.createElement('input');
    input.type='number';input.min='.1';input.max='3600';input.step='.1';
    input.value=s.duration;input.disabled=running;
    input.onchange=()=>{
      s.duration=Math.max(.1,Math.min(3600,Number(input.value)||.1));
      render();
    };
    editor.appendChild(input);
    const info=document.createElement('small');
    info.textContent=' Duration in seconds';
    editor.appendChild(info);
  }
}

function dropOnStep(targetId){
  if(running||!dragData)return;
  const target=steps.findIndex(s=>s.id===targetId);
  if(target<0)return;

  if(dragData.kind==='new'){
    addStep(dragData.type,target);
  }else{
    const from=steps.findIndex(s=>s.id===dragData.id);
    if(from<0||from===target){dragData=null;return;}
    const [item]=steps.splice(from,1);
    const adjusted=from<target?target-1:target;
    steps.splice(adjusted,0,item);
    selectedId=item.id;
    render();
  }
```

```
  dragData=null;
}

document.querySelectorAll('.action').forEach(button=>{
  button.onclick=()=>addStep(button.dataset.type);
  button.addEventListener('dragstart',e=>{
    if(running){e.preventDefault();return;}
    dragData={kind:'new',type:button.dataset.type};
    e.dataTransfer.setData('text/plain','new:'+button.dataset.type);
  });
});

lane.addEventListener('dragover',e=>e.preventDefault());
lane.addEventListener('drop',e=>{
  e.preventDefault();
  if(running||!dragData)return;
  if(dragData.kind==='new')addStep(dragData.type);
  else{
    const from=steps.findIndex(s=>s.id===dragData.id);
    if(from>=0){
      const [item]=steps.splice(from,1);
      steps.push(item);selectedId=item.id;render();
    }
  }
  dragData=null;
});

function duplicateSelected(){
  if(running)return;
  const i=steps.findIndex(s=>s.id===selectedId);
  if(i<0)return;
  const copy={...steps[i],id:nextId++};
  steps.splice(i+1,0,copy);
  selectedId=copy.id;render();
}
function deleteSelected(){
  if(running)return;
  steps=steps.filter(s=>s.id!==selectedId);
  selectedId=null;render();
}
function clearTimeline(){
  if(running)return;
  steps=[];selectedId=null;render();
}
$('turnSeconds').addEventListener('change',()=>{
  const v=Math.max(1,Math.min(120,Number($('turnSeconds').value)||17.1));
  $('turnSeconds').value=v;render();
});

async function request(path){
  try{
    const response=await fetch(path,{cache:'no-store'});
    if(!response.ok)throw new Error('HTTP '+response.status);
    return true;
  }catch(e){
    setStatus('Connection lost. Motor timeout should stop movement.');
    return false;
```

```
  }
}
async function sendStop(){return request('/stop');}
async function sendMove(dir){return request('/move?dir='+dir);}

async function runTimeline(){
  if(running)return;
  if(!steps.length){setStatus('Add actions to the timeline first.');return;}

  running=true;paused=false;stopRequested=false;
  // Snapshot playback so editing/importing cannot change a running sequence.
  const sequence=JSON.parse(JSON.stringify(steps));
  const turnSeconds=Math.max(1,Math.min(120,Number($('turnSeconds').value)||17.1));
  const repeat=$('repeat').checked;
  setStatus(repeat?'Repeating timeline':'Running timeline');
  let completed=false;

  try{
    do{
      for(let i=0;i<sequence.length;i++){
        if(stopRequested)break;
        const s=sequence[i];
        activeId=s.id;render();

        const duration=(s.type==='turn'?turnSeconds:durationOf(s))*1000;
        const moving=['left','right','turn'].includes(s.type);
        const direction=s.type==='turn'?s.direction:s.type;
        let remaining=duration,last=Date.now(),heartbeat=0;

        if(!moving)await sendStop();

        while(remaining>0&&!stopRequested){
          if(paused){
            await sendStop();
            await sleep(80);
            last=Date.now();
            continue;
          }

          const now=Date.now();
          if(moving&&now-heartbeat>=250){
            const ok=await sendMove(direction);
            if(!ok){stopRequested=true;break;}
            heartbeat=Date.now();
          }
          await sleep(40);
          const after=Date.now();
          remaining-=after-last;
          last=after;
          $('progress').textContent='Action '+(i+1)+'/'+sequence.length+
            ' · '+label(s.type)+' · '+Math.max(0,remaining/1000).toFixed(1)+'s left';
        }
        await sendStop();
        if(stopRequested)break;
      }
      if(stopRequested)break;
      completed=true;
```

```
      if(repeat){
        $('progress').textContent='Cycle complete — restarting from the beginning.';
        await sleep(100);
      }
    }while(repeat&&!stopRequested);
  }finally{
    await sendStop();
    running=false;paused=false;activeId=null;render();
    setStatus(stopRequested?'Stopped':(completed?'Timeline complete':'Playback ended'));
    $('progress').textContent=stopRequested?'Playback cancelled.':
      (repeat?'Repeat disabled or playback ended.':'Playback finished.');
  }
}
function sleep(ms){return new Promise(resolve=>setTimeout(resolve,ms));}

function pauseTimeline(){
  if(!running)return;
  paused=!paused;
  if(paused){sendStop();setStatus('Paused — motor stopped');}
  else setStatus('Resuming');
}
function stopTimeline(){
  stopRequested=true;paused=false;sendStop();setStatus('Stopping...');
}

// Browser-local saved sequences
function savedSequences(){
  try{return JSON.parse(localStorage.getItem('cineorbit_sequences')||'{}');}
  catch(e){return {};}
}
function refreshSavedList(selected=''){
  const list=$('savedList');
  list.innerHTML='<option value="">Choose saved sequence</option>';
  Object.keys(savedSequences()).sort().forEach(name=>{
    const o=document.createElement('option');o.value=name;o.textContent=name;
    list.appendChild(o);
  });
  list.value=selected;
}
function saveSequence(){
  if(running)return;
  const name=$('sequenceName').value.trim();
  if(!name){setStatus('Enter a sequence name first.');return;}
  const all=savedSequences();
  all[name]={version:2,steps,turnSeconds:$('turnSeconds').value};
  localStorage.setItem('cineorbit_sequences',JSON.stringify(all));
  refreshSavedList(name);setStatus('Saved: '+name);
}
function loadSequence(){
  if(running)return;
  const name=$('savedList').value, data=savedSequences()[name];
  if(!data){setStatus('Choose a saved sequence first.');return;}
  if(!validateSequence(data)){setStatus('Saved sequence is invalid.');return;}
  steps=data.steps.map(s=>({...s,id:nextId++}));
  selectedId=null;
  $('sequenceName').value=name;
  $('turnSeconds').value=data.turnSeconds||17.1;
```

```
  render();setStatus('Loaded: '+name);
}
function deleteSequence(){
  if(running)return;
  const name=$('savedList').value;
  if(!name)return;
  const all=savedSequences();delete all[name];
  localStorage.setItem('cineorbit_sequences',JSON.stringify(all));
  refreshSavedList();setStatus('Deleted: '+name);
}

// JSON file export/import
function exportJSON(){
  const name=$('sequenceName').value.trim()||'CineOrbit-sequence';
  const data={
    app:'CineOrbit',
    version:2,
    name,
    turnSeconds:Number($('turnSeconds').value)||17.1,
    steps:steps.map(({type,duration,direction})=>({
      type,duration,direction
    }))
  };
  const blob=new Blob([JSON.stringify(data,null,2)],{type:'application/json'});
  const url=URL.createObjectURL(blob);
  const a=document.createElement('a');
  a.href=url;a.download=name.replace(/[^a-z0-9_-]/gi,'_')+'.json';
  document.body.appendChild(a);a.click();a.remove();
  URL.revokeObjectURL(url);
  setStatus('JSON exported.');
}

function validateSequence(data){
  if(!data||!Array.isArray(data.steps)||data.steps.length>500)return false;
  if(data.steps.some(s=>{
    if(!s||!['left','right','stop','turn'].includes(s.type))return true;
    if(s.type==='turn')return !['left','right'].includes(s.direction);
    return !Number.isFinite(Number(s.duration))||
      Number(s.duration)<.1||Number(s.duration)>3600;
  }))return false;
  const t=Number(data.turnSeconds||17.1);
  return Number.isFinite(t)&&t>=1&&t<=120;
}

$('importFile').addEventListener('change',async e=>{
  const file=e.target.files[0];
  e.target.value='';
  if(!file)return;
  if(running){setStatus('Stop playback before importing.');return;}
  try{
    const data=JSON.parse(await file.text());
    if(!validateSequence(data))throw new Error('Invalid sequence');
    steps=data.steps.map(s=>({
      id:nextId++,type:s.type,
      duration:Number(s.duration)||3,
      direction:s.direction||'right'
    }));
```

```
    $('turnSeconds').value=Number(data.turnSeconds)||17.1;
    $('sequenceName').value=data.name||file.name.replace(/\.json$/i,'');
    selectedId=null;render();setStatus('Imported: '+file.name);
   }catch(err){
    setStatus('Import failed: invalid CineOrbit JSON file.');
   }
});

refreshSavedList();
render();
</script>
</body>
</html>
)rawliteral";

void stopMotor() {
  analogWrite(ENA, 0);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  motorRunning = false;
}

void moveMotor(bool right) {
  digitalWrite(IN1, right ? HIGH : LOW);
  digitalWrite(IN2, right ? LOW : HIGH);
  analogWrite(ENA, MOTOR_SPEED);
  motorRunning = true;
  lastCommandTime = millis();
}

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  analogWriteRange(1023);
  stopMotor();

  Serial.begin(115200);
  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_NAME, AP_PASSWORD);

  Serial.println();
  Serial.print("CineOrbit IP: ");
  Serial.println(WiFi.softAPIP());

  server.on("/", []() {
    server.send_P(200, "text/html", PAGE);
  });

  server.on("/move", []() {
    if (!server.hasArg("dir")) {
      server.send(400, "text/plain", "Missing direction");
      return;
    }
    String dir = server.arg("dir");
    if (dir == "left") moveMotor(false);
    else if (dir == "right") moveMotor(true);
```

```
    else {
      server.send(400, "text/plain", "Invalid direction");
      return;
    }
    server.send(200, "text/plain", "OK");
  });

  server.on("/stop", []() {
    stopMotor();
    server.send(200, "text/plain", "STOPPED");
  });

  server.begin();
}

void loop() {
  server.handleClient();

  if (motorRunning &&
      millis() - lastCommandTime > COMMAND_TIMEOUT) {
    stopMotor();
  }
}
```

OUTPUT

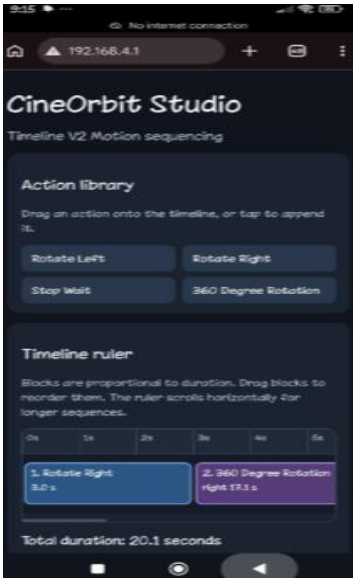


Figure 4.6: The CineOrbit web interface allows users to create and control timed camera-movement sequences



Figure 4.7: The final CineOrbit prototype demonstrates the integration of a smartphone-mounted rotating camera platform with a motorized robotic base for controlled camera movement.

EXPERIMENTAL RESULTS AND ANALYSIS

The RCPLS framework is intended to be evaluated by comparing conventional landmark-selection methods with route-conditioned predictive selection. Performance metrics include landmark tracking duration, landmark loss rate, backup recovery success, tracking continuity and computational time. The results will help determine whether incorporating planned camera movement into landmark selection improves tracking reliability. Since quantitative experimental measurements and completed RCPLS implementation results have not yet been provided, numerical performance improvements cannot currently be established. The final evaluation should include measured results, comparison tables and observations from repeated experiments under different camera trajectories and environmental conditions.

Comparison of Existing Robotic Camera Systems

Existing camera-tracking products can be classified into three categories based on their price, automation capabilities and intended applications.

A. Low-end devices (₹5,000–₹15,000)

Product	Approx. price	Features	Limitations
DJI Osmo Mobile 7	₹6,999	3-axis stabilization, ActiveTrack 7.0, smartphone tracking	No independent mobile rover or programmable dual-motion timeline
DJI Osmo Mobile 7P	₹9,590	Subject tracking, extension rod, tripod, lighting module	Primarily designed for handheld smartphone filming

.

B. Medium-end devices (₹20,000–₹1,00,000)

Product	Approx. price	Features	Limitations
DJI RS 4 Mini	₹29,990	3-axis camera stabilization, up to 2 kg payload, optional intelligent tracking module	Requires a separate camera operator or compatible tracking setup
OBSBOT Tail Air	₹54,980	4K video, AI subject tracking, 2-axis gimbal, 320° pan, wireless connectivity	Primarily a stationary PTZ camera; no mobile rover platform

.

C. High-end devices (above ₹1,00,000)

Product	Approx. price	Features	Limitations
MOVE'N SEE PIXEM 2	₹1,98,540	Automatic subject following, wireless tracking watch, indoor/outdoor operation, 360° rotation	Relies on a dedicated wearable tracking beacon
SalrayWorks N BOT NDI	₹4,22,325	Robotic pan/tilt head, preset positions, 0.1° positioning accuracy, NDI connectivity	Primarily intended for professional fixed-camera installations

.

Comparative Analysis and Research Positioning

The comparison indicates that low-end devices mainly focus on smartphone stabilization and basic subject tracking, whereas medium-end products offer advanced tracking, camera stabilization and automated pan/tilt capabilities. High-end systems provide professional robotic camera operation, sophisticated tracking mechanisms and precise positioning, but involve significantly higher investment. CineOrbit aims to occupy a different position by combining a programmable rotating camera platform, a mobile rover, independent parallel movement timelines and the proposed RCPLS vision framework in a low-cost system. Unlike the compared commercial products, its intended architecture combines camera rotation and ground mobility in one platform. Its estimated prototype cost is approximately ₹2,500, based on the project's stated budget. However, this is a preliminary cost comparison, not evidence that CineOrbit currently matches the tracking accuracy, video quality or reliability of commercial systems. Experimental validation of RCPLS and the complete robotic platform is required to establish its actual performance and advantages.

CONCLUSION

CineOrbit presents a low-cost robotic cinematography platform designed to automate camera movement through the integration of embedded systems, wireless communication and computer vision. The system combines a rotating camera mechanism with a mobile rover, enabling independent control of camera rotation and ground movement. Its web-based interface provides a programmable timeline editor through which users can define, organize and execute camera and rover movements simultaneously. The ESP32 serves as the central controller, coordinating motor operations through L298N motor drivers and supporting additional hardware such as an IR obstacle-detection sensor.

A key aspect of the proposed system is Route-Conditioned Predictive Landmark Selection (RCPLS), a framework intended to improve visual tracking by incorporating planned camera trajectories into landmark selection. By combining established feature-detection and tracking techniques with predictive landmark ranking and backup management, the framework aims to maintain reliable visual references during camera movement. The project demonstrates an approach to making robotic cinematography more accessible and affordable, with an estimated prototype cost of approximately ₹2,500. Overall, CineOrbit provides a foundation for developing programmable, autonomous and cost-effective robotic camera systems.

FUTURE SCOPE

In summary, a well-maintained database of alumni connections can have a positive and far-reaching impact on society by fostering networking, mentorship, career development community-building while also supporting educational institutions and research effort. It serves as a valuable resource for graduates, students, and the institutions themselves, contributing to a more interconnected and informed society.

REFERENCES

1. Shi, J., and Tomasi, C. (1994). “Good Features to Track.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 593–600.
2. Rublee, E., Rabaud, V., Konolige, K., and Bradski, G. (2011). “ORB: An Efficient Alternative to SIFT or SURF.” *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2564–2571. https://doi.org/10.1109/ICCV.2011.6126544
3. Lucas, B. D., and Kanade, T. (1981). “An Iterative Image Registration Technique with an Application to Stereo Vision.” *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 674–679.
4. Horn, B. K. P., and Schunck, B. G. (1981). “Determining Optical Flow.” *Artificial Intelligence*, 17(1–3), pp. 185–203. https://doi.org/10.1016/0004-3702(81)90024-2
5. Chaumette, F., and Hutchinson, S. (2006). “Visual Servo Control. I. Basic Approaches.” *IEEE Robotics & Automation Magazine*, 13(4), pp. 82–90. https://doi.org/10.1109/MRA.2006.250573
6. Espressif Systems. *ESP32 Series Datasheet*. Available at: https://www.espressif.com/
7. OpenCV. *OpenCV Documentation*. Available at: https://docs.opencv.org/
8. OpenCV.js. *OpenCV.js Documentation*. Available at: https://docs.opencv.org/4.x/d5/d10/tutorial_js_root.html

QR CODE



https://github.com/vrak-akshay/Cineorbit